



SOLR

an Open-source REST-API based search server platform

Academic Year 2025/26

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
Master Degree in Computer Engineering

Installing Solr

What is Solr



- A search platform
- An Open Source project developed by Apache Foundation
- It is written in Java (you need JDK or JRE 8)
- It can be used as a Search Engine (like Google or Bing)
- It can handle both structured and unstructured data
- It can crawl and index data from web page
- It is able to run also semantic search

What is Solr



- Powerful, scalable and secure
- It was developed in 2004
- Recently, version 9.5.0 has been released
- Queries are transmitted to Solr via HTTP 1.1 or 2.0 requests
- Response is typically a list of structured document descriptors
- Response can be coded into different standard: JSON, CSV, PHP
- Any platform capable of HTTP can talk to Solr

How to install Solr



- Download the project from <https://solr.apache.org/>
 - `wget https://www.apache.org/dyn/closer.lua/solr/solr/9.5.0/solr-9.5.0.tgz`
- Unzip the package
 - `ls -l solr-9.5.0.tgz`
 - `tar -xvzf solr-9.5.0.tgz`
 - `cd solr-9.5.0`
- Nothing else: ready to use

Inside the Solr package



bin: contains all binaries used to start and stop the server

server: contains configuration files, log files and scripts

Terminology



- **Document:** anything containing data. A document is a collection of fields
- **Field:** the content you want to index or search. The minimal information to manage
- **Field Type:** the type of each field
- **Schema:** Fields and Field Types used in a collection
- **Posting/Ingestion:** the process where data are stored into the database and then are indexed to be retrieved quickly

Terminology



- **Node:** the server where Solr is running on
- **Instance:** the job executed on the physical server. There might be several instances on the same server, all of them binded on different ports
- **Standalone Mode:** a configuration with a single solr instance running on a single node
- **Cluster Mode:** a configuration with many solr instances running on several nodes

Terminology



- **Core/Collection:** a repository containing documents (core for standalone, collection for cluster)
- **Shard:** a logical partition of a collection. A collection can be split in one or more shards
- **Replica:** a process to distribute data across several nodes, usually used for high availability and fault tolerance

Start and stop Solr in standalone mode



```
$ bin/solr start
```

The server is started on the 8983 port by default

```
$ bin/solr status
```

```
$ bin/solr stop
```

Structured and unstructured data



Solr is able to manage two different sets of data:

- **structured data:** data that have a regular and predefined structured (similar to those data stored in a relational database)
- **unstructured data:** for example web pages

Structured data



Let's have a look to the file:

- `films_small.csv`

Six fields:

- `id`
- `name`
- `directed_by`
- `genre` (multi value attribute)
- `type`
- `initial_release_date`

Managing data

How to interact to Solr



Solr can be managed using three different interfaces:

- a minimal web interface: <http://localhost:8983/solr>
- a powerful command line
 - `bin/solr`: to interact with the server
 - `bin/post`: to ingest documents
- `curl`: external application can use HTTP/HTTPS for indexing and searching data

The User Interface

The screenshot displays the Solr Admin web interface in a browser window. The address bar shows `localhost:8983/solr/#/`. The interface is divided into a left sidebar, a main content area, and a right-hand system status panel.

Left Sidebar: Contains navigation links: **Dashboard** (highlighted with a red box), Logging, Security, Core Admin, Java Properties, Thread Dump, and a message "No cores available Go and create one".

Main Content Area:

- Instance:** Shows "Start" as "less than a minute ago".
- Versions:** A table listing installed versions. The version **9.5.0** is highlighted with a red box. The table includes columns for version, spec, impl, and a timestamp.
- JVM:** Shows "Runtime" as "Oracle Corporation Java HotSpot(TM) 64-Bit Server VM 15.0.1 15.0.1+9-18".
- Processors:** The value **8** is highlighted with a red box.
- Args:** A list of system arguments, including `-DSTOP.KEY=solrlocks`, `-DSTOP.PORT=7983`, `-DdisableAdminUI=false`, `-Dhost=localhost`, `-Djava.security.manager`, `-Djava.security.policy=/usr/local/solr-9.5.0/server/etc/security.policy`, `-Djava.security.properties=/usr/local/solr-9.5.0/server/etc/security.properties`, `-Djava.util.logging.manager=org.apache.logging.log4j.jul.LogManager`, `-Djetty.home=/usr/local/solr-9.5.0/server`, `-Djetty.port=8983`, `-Dsolr.default.confdir=/usr/local/solr-9.5.0/server/solr/configsets/_default/conf`, `-Dsolr.install.dir=/usr/local/solr-9.5.0`, `-Dsolr.install.symDir=/usr/local/solr-9.5.0`, `-Dsolr.internal.network.permission=*`, and `-Dsolr.jetty.inetaccess.excludes=*`.

Right-Hand System Status Panel: A red box highlights the "System" and "JVM-Memory" sections.

- System:** Shows overall system metrics: Physical Memory (87.4%), Swap Space (0.1%), File Descriptor Count (0.1%), and JVM-Memory (30.2%). Each metric is accompanied by a horizontal bar chart showing current usage versus total capacity.
- Security:** A warning message states: "WARNING: Security is not enabled for this server! See security screen for how to enable authentication TLS enabled? ✗".

Creating a core through GUI



Dashboard

Logging

Security

Core Admin

Java Properties

Thread Dump

No cores available
Go and create one

+ Add Core

name: new_core

instanceDir: new_core

dataDir: data

config: solrconfig.xml

schema: schema.xml

i instanceDir and dataDir need to exist before you can create the core

✓ Add Core

✗ Cancel

1. Select Core Admin

2. Fill the name of the core we want to add (for example films_small)

3. Set the instanceDir. This directory must exist before creating the core. To do so:

```
$ cp -r server/solr/configsets/_default  
server/solr/new_core
```


Creating a core through UI

The screenshot displays the Solr Admin UI for a core named 'myFirstCore'. The interface is divided into three main sections, each highlighted with a red circle and a number:

- 1. Core Selection:** The 'myFirstCore' core is selected in the left-hand navigation menu.
- 2. Statistics:** The 'Statistics' tab shows the following data:
 - Last Modified: -
 - Num Docs: 0
 - Max Doc: 0
 - Deleted Docs: 0
 - Version: 2
 - Segment Count: 0
 - Current: ✓
- 3. Instance:** The 'Instance' tab shows the following configuration:
 - CWD: /usr/local/solr-9.5.0/server
 - Instance: /usr/local/solr-9.5.0/server/solr/myFirstCore
 - Data: /usr/local/solr-9.5.0/server/solr/myFirstCore/data
 - Index: /usr/local/solr-9.5.0/server/solr/myFirstCore/data/index
 - Impl: org.apache.solr.core.NRTCachingDirectoryFactory

Below the 'Statistics' tab, the 'Replication (Leader)' section shows a table with the following data:

	Version	Gen	Size
Leader (Searching)	0	1	69 bytes
Leader (Replicable)	-	-	-

On the left sidebar, the navigation menu includes: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, myFirstCore (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, and Query.

1. From the Selector Core, select the core

2. Some statistics about the number of documents indexed or deleted

- **numDocs:** number of searchable documents in the index
- **maxDoc:** includes logically deleted documents that have not yet been physically removed from the index

3. Some information about the path of the data and the index

Creating a core by using Command Line



```
$ bin/solr create -c films_small
```

This command creates a core named films. The core is empty, no document is stored inside

Creating a core: under the hood



1. a new directory named with the core name is created under server/solr
2. this directory contains two subdirs: data and conf
3. data contains documents and indexes
4. conf contains the core configuration. Such configuration is the default, as a copy of server/solr/configsets/_default directory

BE AWARE: creating a core with the default configuration is not always good.

During core creation, the systems warns you:

WARNING: Using _default configset with data driven schema functionality. NOT RECOMMENDED for production use.

Posting new documents

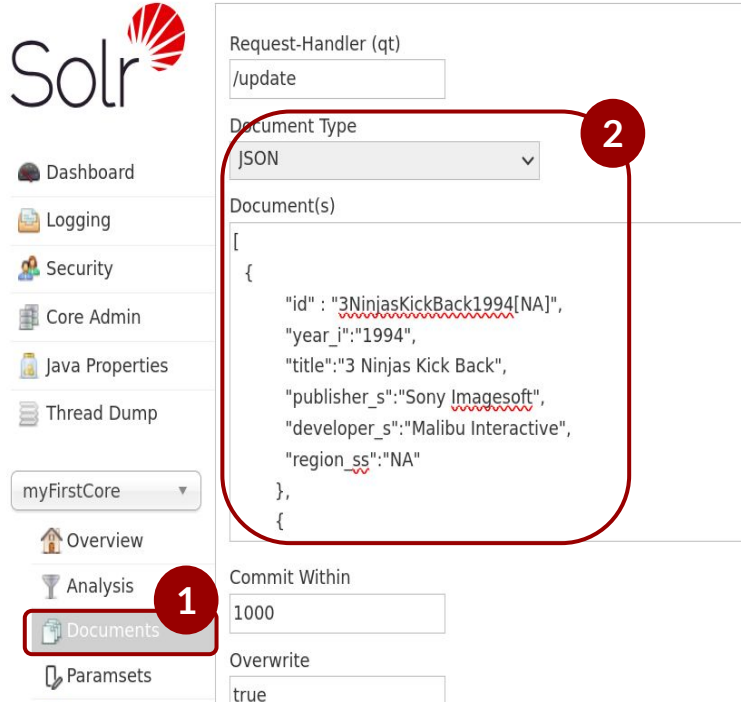


Solr can ingest many types of files such as XML, CSV, JSON.

For different formats (PDF, Doc, HTML) Solr makes use of Solr Cell (a framework based on Apache Tika).

Posting can be done using User Interface or Command Line

Posting new documents using UI



The screenshot shows the Solr Admin UI. On the left is a sidebar with navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, myFirstCore (selected), Overview, Analysis, Documents (highlighted with a red box and a red circle with the number 1), and Paramsets. The main content area is titled 'Request-Handler (qt)' and shows the following fields: 'Document Type' set to 'JSON' (highlighted with a red box and a red circle with the number 2), 'Document(s)' containing a JSON object, 'Commit Within' set to 1000, and 'Overwrite' set to true. The JSON object is:

```
{
  "id" : "3NinjasKickBack1994[NA]",
  "year_i": "1994",
  "title": "3 Ninjas Kick Back",
  "publisher_s": "Sony Imagesoft",
  "developer_s": "Malibu Interactive",
  "region_ss": "NA"
},
{
```

1. From the Selector Core, select the core you want to add documents to

2. If your documents are in JSON, select the correct document type, then paste the content into the box

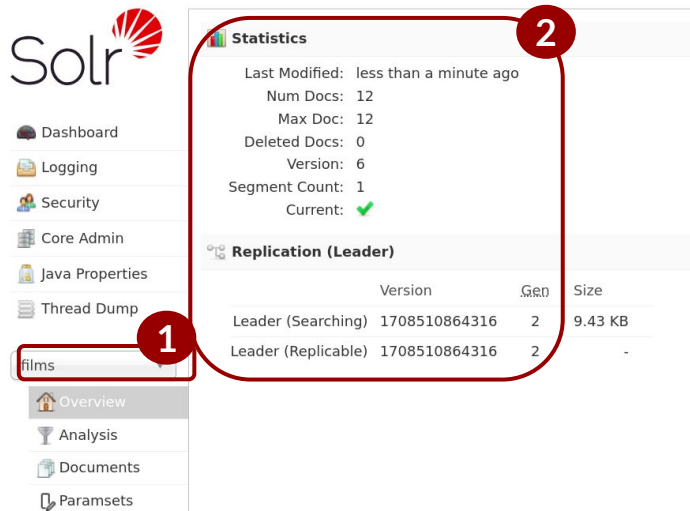
Posting new documents using CL



```
$ bin/post -c films example/films/films_small.csv
```

films_small.csv contains 12 films

Posting new documents using CL



The screenshot shows the Solr Admin interface. On the left is a sidebar with navigation links: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, films (highlighted with a red circle and the number 1), Overview, Analysis, Documents, and Paramsets. The main content area displays the 'Statistics' for the 'films' core (highlighted with a red circle and the number 2). The statistics show: Last Modified: less than a minute ago, Num Docs: 12, Max Doc: 12, Deleted Docs: 0, Version: 6, Segment Count: 1, and Current: with a green checkmark. Below the statistics is the 'Replication (Leader)' section, which contains a table with the following data:

	Version	Gen	Size
Leader (Searching)	1708510864316	2	9.43 KB
Leader (Replicable)	1708510864316	2	-

1. The core now contains 12 documents

Let's browse data using UI

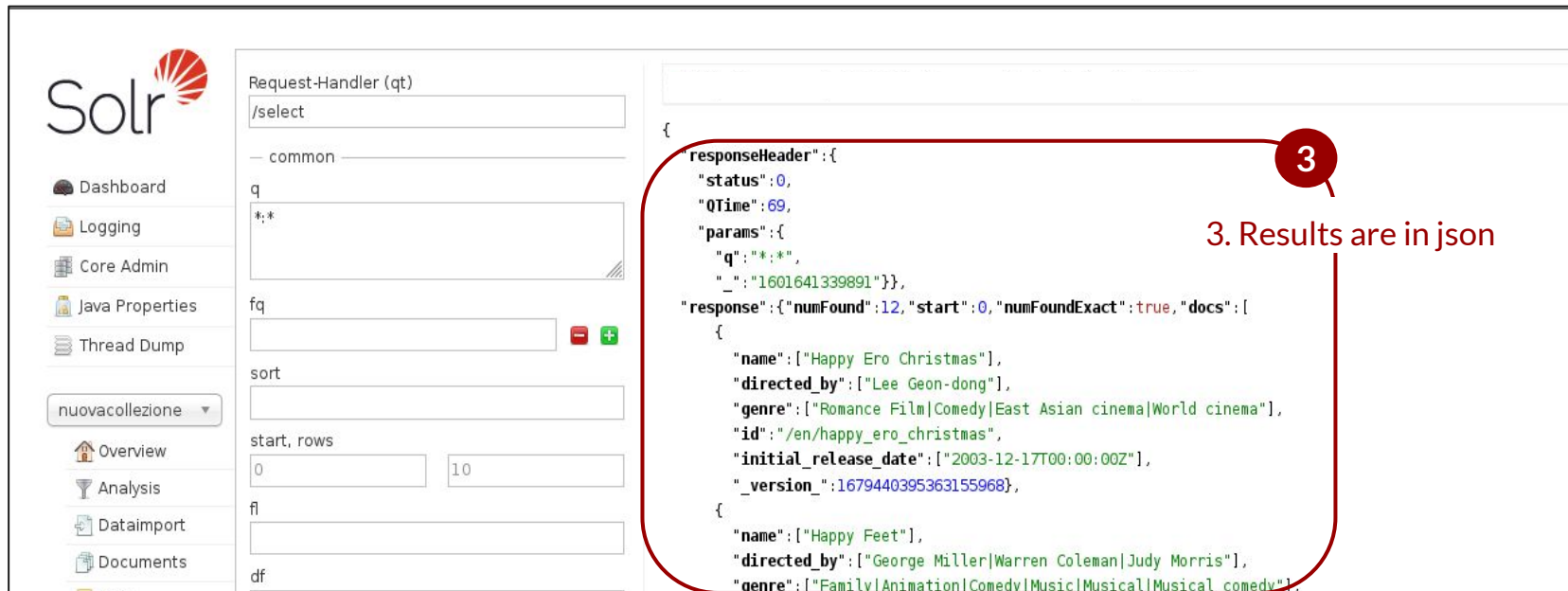
The screenshot shows the Solr Admin UI. On the left is a sidebar with a menu. The 'Query' option at the bottom is highlighted with a red circle and a red number '1'. The main area shows the 'Request-Handler (qt)' configuration form, which is also outlined with a red circle and a red number '2'. The form contains the following fields:

- Request-Handler (qt): /select
- common: (empty)
- q: **
- q.op: OR (dropdown)
- fq: (empty)
- sort: (empty)
- start, rows: 0 (start), 10 (rows)
- fl: (empty)
- df: (empty)
- paramset(s): Select paramset(s)...
- wt: ----- (dropdown)

1. Select Query

2. For now, leave all fields empty

Let's browse data

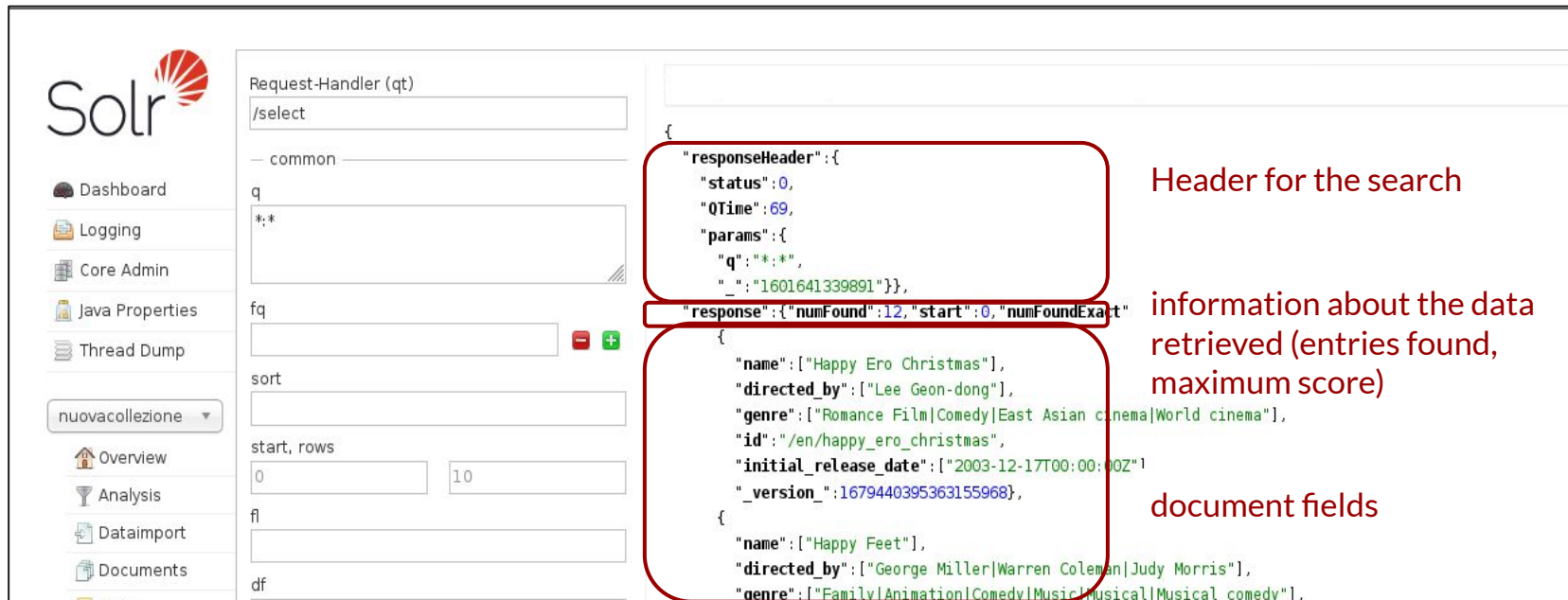


The screenshot displays the Solr Admin interface. On the left is a sidebar with navigation links: Dashboard, Logging, Core Admin, Java Properties, Thread Dump, a dropdown menu (nuovacollezione), Overview, Analysis, Dataimport, and Documents. The main area is titled 'Request-Handler (qt)' and contains a search form with fields for 'q' (query), 'fq' (facet query), 'sort', 'start, rows', 'fl' (fields list), and 'df' (distribution fields). The 'q' field contains the query '*:*'. The 'start, rows' field shows '0' and '10'. The 'fl' field is empty. The 'df' field is empty. The 'Response' field on the right displays the JSON output of the search. A red circle with the number '3' highlights the 'response' field in the JSON, with a red arrow pointing to it and the text '3. Results are in json'.

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 69,
    "params": {
      "q": "*:*",
      "_": "1601641339891"
    }
  },
  "response": {
    "numFound": 12,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "name": ["Happy Ero Christmas"],
        "directed_by": ["Lee Geon-dong"],
        "genre": ["Romance Film|Comedy|East Asian cinema|World cinema"],
        "id": "/en/happy_ero_christmas",
        "initial_release_date": ["2003-12-17T00:00:00Z"],
        "_version_": 1679440395363155968
      },
      {
        "name": ["Happy Feet"],
        "directed_by": ["George Miller|Warren Coleman|Judy Morris"],
        "genre": ["Family|Animation|Comedy|Music|Musical|Musical comedy"]
      }
    ]
  }
}
```

3. Results are in json

Let's browse data



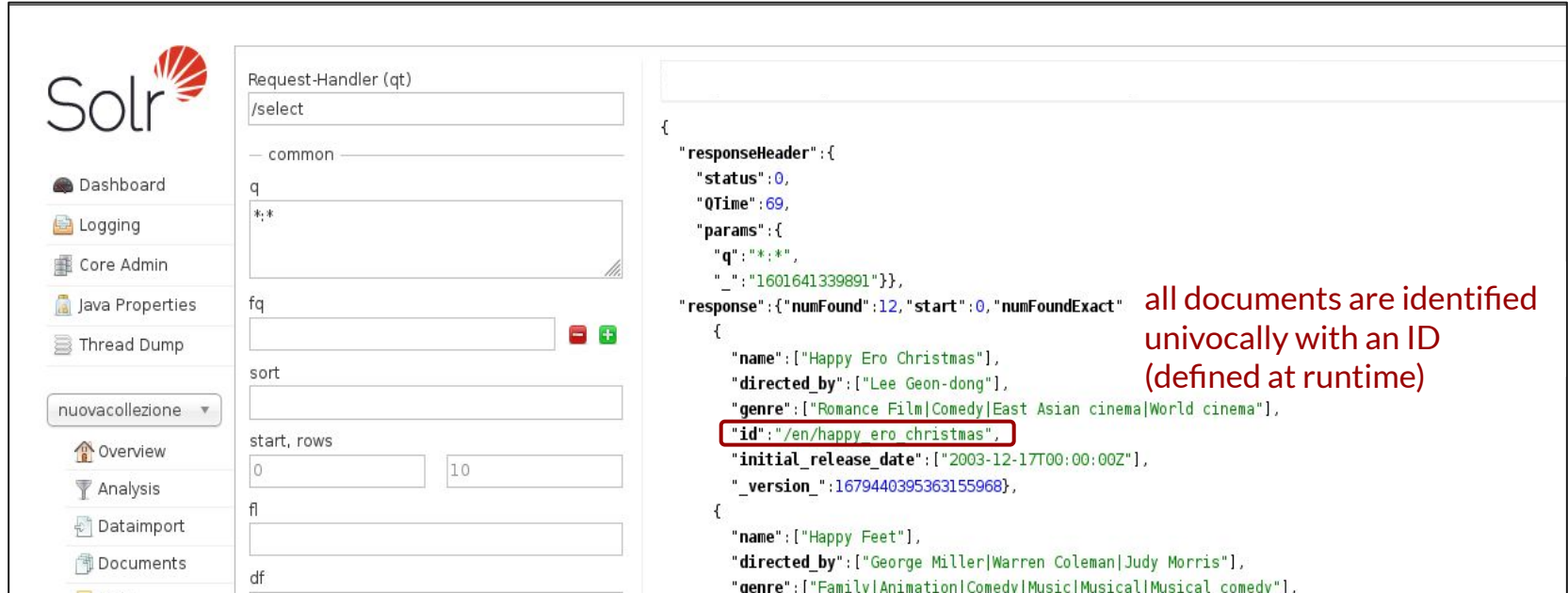
The screenshot displays the Solr Admin interface. On the left is a sidebar with navigation links: Dashboard, Logging, Core Admin, Java Properties, Thread Dump, and a dropdown menu for 'nuovacollezione' containing Overview, Analysis, Dataimport, and Documents. The main area is titled 'Request-Handler (qt)' and shows a search request with the following parameters:

- Request: `/select`
- Common: `— common —`
- Query: `q: *,*`
- Facet: `fq: [ ]`
- Sort: `sort: [ ]`
- Start/Rows: `start, rows: 0 10`
- Filter: `fl: [ ]`
- Distribution: `df: [ ]`

The JSON response is displayed on the right, with two sections highlighted by red boxes and labeled:

- Header for the search:** The `"responseHeader"` section, which includes:
 - `"status": 0`
 - `"QTime": 69`
 - `"params": { "q": "*,*", "_": "1601641339891" }`
- information about the data retrieved (entries found, maximum score):** The `"response"` section, which includes:
 - `"numFound": 12`
 - `"start": 0`
 - `"numFoundExact": true`
- document fields:** The array of document objects, each containing fields like `"name"`, `"directed_by"`, `"genre"`, `"id"`, `"initial_release_date"`, and `"_version_"`.

Let's browse data



The screenshot displays the Solr Admin interface. On the left is a sidebar with navigation links: Dashboard, Logging, Core Admin, Java Properties, Thread Dump, and a dropdown menu 'nuovacollezione' containing Overview, Analysis, Dataimport, and Documents. The main area is titled 'Request-Handler (qt)' and shows a search query `/select` with a 'common' filter. The search results are displayed in a table with columns for 'fq', 'sort', 'start, rows', 'fl', and 'df'. The first result is for the document with ID `1601641339891`, which is highlighted with a red box. The JSON response for this document is shown on the right, with the `"id": "/en/happy_ero_christmas"` field also highlighted with a red box. A red text annotation points to this field, stating: 'all documents are identified univocally with an ID (defined at runtime)'.

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 69,
    "params": {
      "q": "*",
      "_: "1601641339891"
    }
  },
  "response": {
    "numFound": 12,
    "start": 0,
    "numFoundExact": true,
    "docs": [
      {
        "name": "Happy Ero Christmas",
        "directed_by": "Lee Geon-dong",
        "genre": "Romance Film|Comedy|East Asian cinema|World cinema",
        "id": "/en/happy_ero_christmas",
        "initial_release_date": "2003-12-17T00:00:00Z",
        "_version_": 1679440395363155968
      },
      {
        "name": "Happy Feet",
        "directed_by": "George Miller|Warren Coleman|Judy Morris",
        "genre": "Family|Animation|Comedy|Music|Musical|Musical comedy"
      }
    ]
  }
}
```

Let's browse data: observation



1. Data are retrieved and provided in different formats
2. The protocol used to contact Solr is HTTP
 - a. This makes all clients (written in any language) able to get data without any further library

Deleting Data



Delete only the document having ID=SP2514N

```
$ echo "<delete><id>SP2514N</id></delete>" > delete.xml
```

```
$ bin/solr post -c films_small delete.xml
```

Delete all documents

```
$ echo "<delete><query>*:*</query></delete>" > delete.xml
```

```
$ bin/solr post -c films_small delete.xml
```

Query Parser



When a query is submitted, it is managed by a **Query Parser** that is responsible to interpret the parameters sent within the query.

There are about 25 other parsers available for special needs offering flexibility, but three of them are used most often, **Lucene**, **DisMax** and **EDisMax** (Enhanced DisMax).

Searching for Documents



During inquiring, the Query Parser can receive:

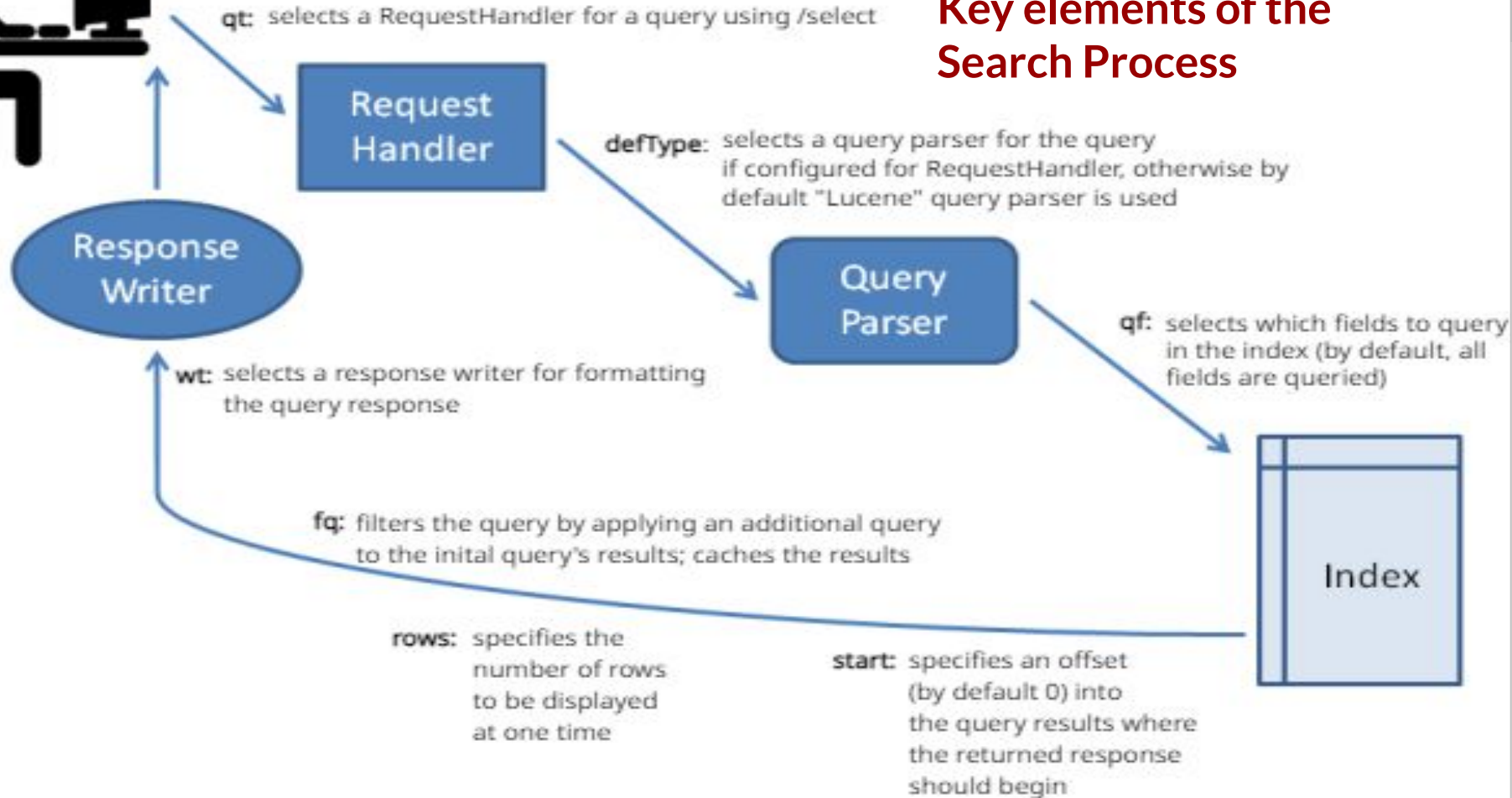
1. Terms to search for
2. Parameters for fine-tuning the query, such as
 - a. parameters to increase the importance of particular fields
 - b. parameters to exclude some content from the result
 - c. boolean logic to apply among terms
3. Parameters for controlling the presentation of the query response
 - a. ordering
 - b. limiting
 - c. list of fields to be retrieved

Searching for Documents



4. Parameters for highlighting terms
5. Parameters for including snippets (document excerpts)
6. Parameters for faceting
 - a. it shows results into categories based on indexed terms, for each category the number of hits for relevant term is shown
7. Parameters for clustering
 - a. it shows results into clusters depending on the similarity

Key elements of the Search Process



Searching for documents

Logging

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Request-Handler (qt)

/select

— common —

q

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

wt

☐ indent off

The request handler: the component in charge of handling requests. We'll see more details later

Searching for documents

Logging

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Request-Handler (qt)

/select

— common —

q

:

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

wt

☐ indent off

Term I want to search for on ALL document fields

Syntax:
name_field:term

For example:
name:"Happy"
name:"Happy Feet"

* means "all fields" or "all terms"

I can also search for the parameters in two or more fields. This can be achieved using **copy fields** (see later)

Searching for documents

Logging

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Request-Handler (qt)

/select

— common —

q

:

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

wt

☐ indent off

Filtering data. The syntax is the same of the q parameter. For example if I want to filter all movies whose genre is fantasy, I should use **genre:fantasy**

Searching for documents

The screenshot shows a web interface with a sidebar on the left containing various navigation options: Logging, Core Admin, Java Properties, Thread Dump, a dropdown menu with 'films' selected, Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (highlighted), Replication, Schema, and Segments info. The main content area is titled 'Request-Handler (qt)' and contains several input fields: a text field with '/select', a dropdown menu with 'common', a text area with 'q' and '*: *', a text field with 'fq' and red and green buttons, a text field with 'sort' (highlighted with a red box), a 'start, rows' section with input fields for '0' and '10', a text field with 'fl', a text field with 'df', a 'Raw Query Parameters' section with a text field containing 'key1=val1&key2=val2', a dropdown menu with 'wt' and a '▼' arrow, and a checkbox labeled 'indent off'.

Sorting: ASC or DESC

Not all fields are sortable, it depends on the schema

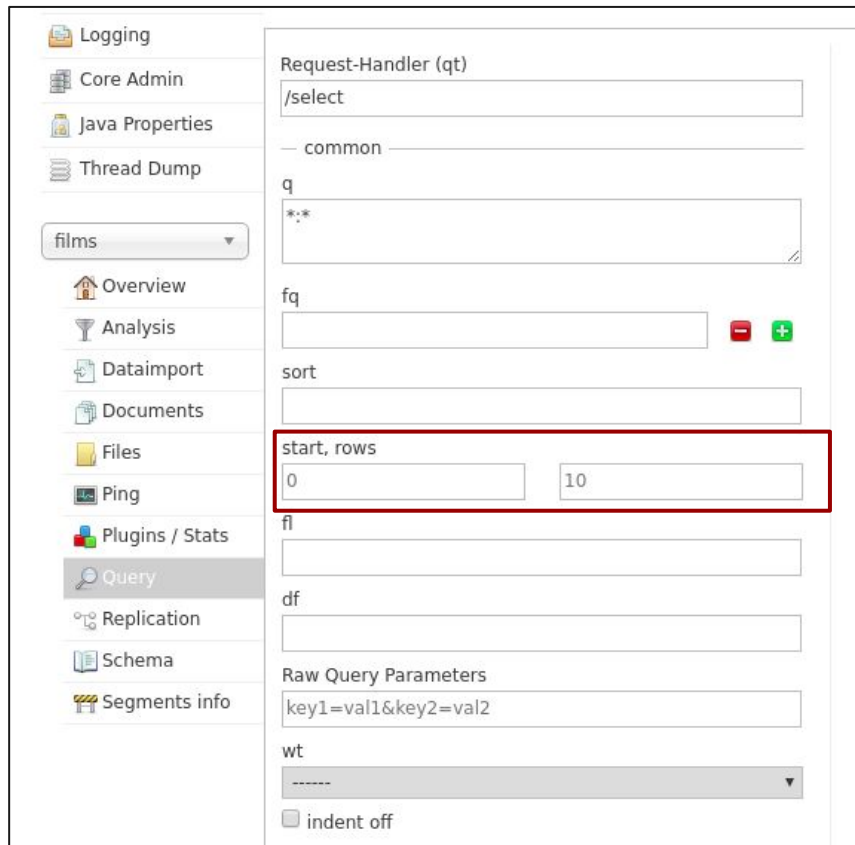
For an id-based sorting:

id asc

or

id desc

Searching for documents



The screenshot shows a web-based interface for searching documents. On the left is a sidebar with a list of navigation items: Logging, Core Admin, Java Properties, Thread Dump, a dropdown menu currently showing 'films', Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (which is highlighted), Replication, Schema, and Segments info. The main panel on the right is titled 'Request-Handler (qt)' and contains several input fields and controls. A red rectangular box highlights the 'start, rows' section, which includes two input fields: the first contains '0' and the second contains '10'. Other visible fields include 'Request-Handler (qt)' with '/select', a 'common' section, a 'q' field with '*:~', an 'fq' field with a red minus and green plus button, a 'sort' field, a 'fl' field, a 'df' field, a 'Raw Query Parameters' field with 'key1=val1&key2=val2', a 'wt' dropdown menu, and an 'indent off' checkbox.

Logging

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Request-Handler (qt)

/select

— common —

q

*:~

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

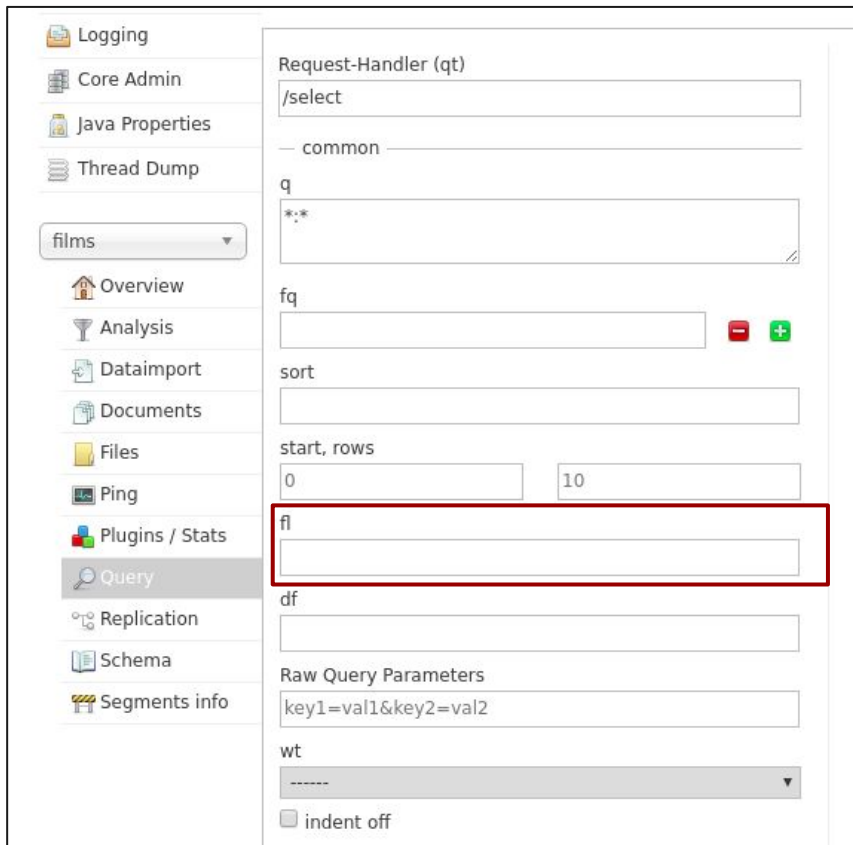
wt

☐ indent off

start: number of document to start from

rows: number of documents retrieved from the document in start

Searching for documents



The screenshot shows the Elasticsearch Query DSL interface. On the left is a sidebar with navigation links: Logging, Core Admin, Java Properties, Thread Dump, a dropdown menu with 'films' selected, Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (highlighted), Replication, Schema, and Segments info. The main panel is titled 'Request-Handler (qt)' and contains several input fields: a text box with '/select', a section '— common —' with a text box containing '*:*', a text box for 'fq' with red minus and green plus icons, a text box for 'sort', a 'start, rows' section with input boxes for '0' and '10', a text box for 'fl' which is highlighted with a red rectangle, a text box for 'df', a 'Raw Query Parameters' section with a text box containing 'key1=val1&key2=val2', a 'wt' dropdown menu, and a checkbox for 'indent off'.

list of field to retrieve, separated by comma.

We can also use a special field, named **score**, that shows how much the documents are relevant according to the search

Searching for documents

Logging

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Request-Handler (qt)

/select

— common —

q

*:~

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

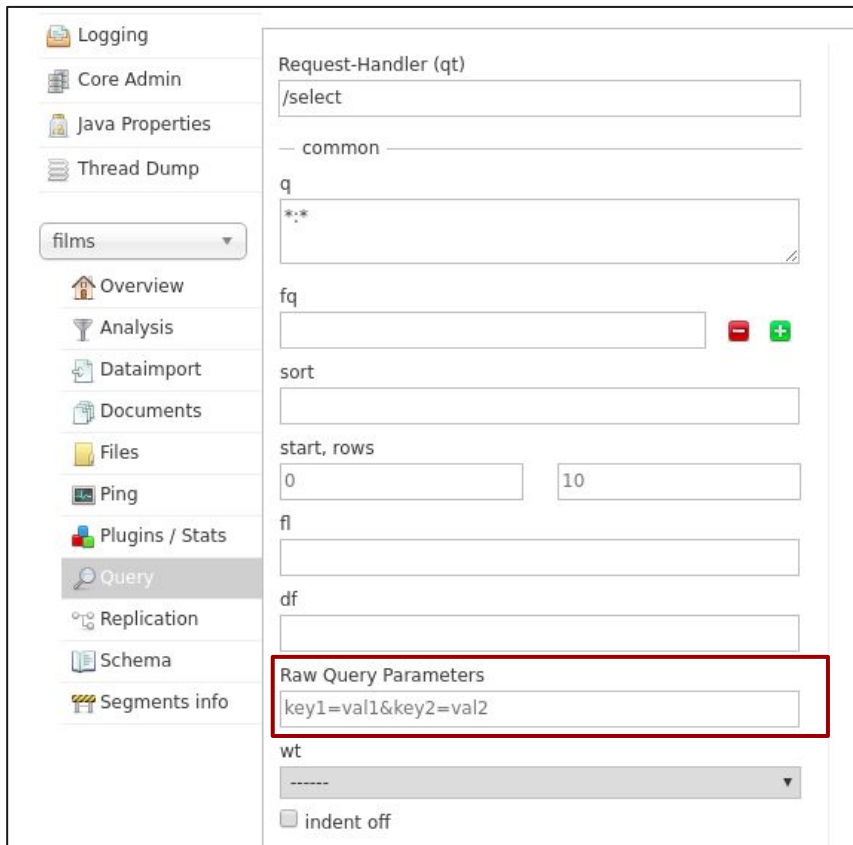
wt

☐ indent off

default field where search is executed when field name is not specified in q parameter.

By default is `_text_` (see further details later)

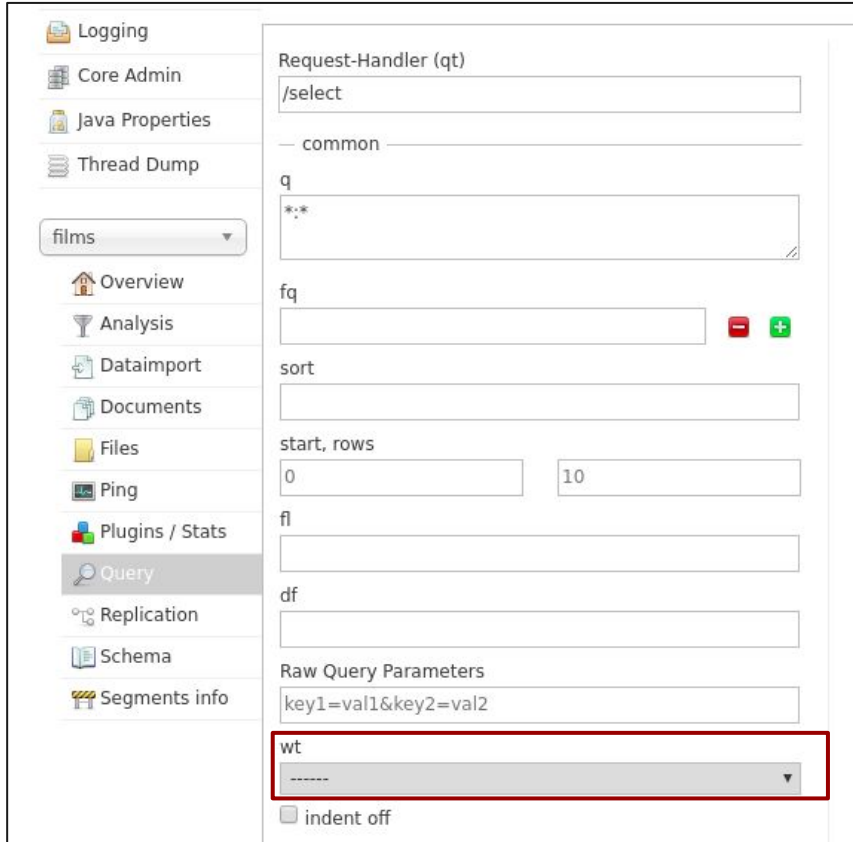
Searching for documents



The screenshot shows the Elasticsearch Query DSL interface. On the left is a sidebar with navigation links: Logging, Core Admin, Java Properties, Thread Dump, a 'films' dropdown, Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (highlighted), Replication, Schema, and Segments info. The main area contains various query parameters: 'Request-Handler (qt)' with a text box containing '/select'; a 'common' section; 'q' with a text box containing '*:*'; 'fq' with a text box and red and green buttons; 'sort' with a text box; 'start, rows' with two text boxes containing '0' and '10'; 'fl' with a text box; 'df' with a text box; 'Raw Query Parameters' (highlighted with a red box) with a text box containing 'key1=val1&key2=val2'; 'wt' with a dropdown menu showing '-----'; and an 'indent off' checkbox.

Raw Query Parameters: used for debugging:
omitHeader=true (cut header in responses)

Searching for documents



The screenshot shows the Elasticsearch Query DSL interface. On the left is a sidebar with navigation links: Logging, Core Admin, Java Properties, Thread Dump, a 'films' dropdown, Overview, Analysis, Dataimport, Documents, Files, Ping, Plugins / Stats, Query (highlighted), Replication, Schema, and Segments info. The main panel contains the following fields:

- Request-Handler (qt):** A text box containing `/select`.
- common:** A section header.
- q:** A text box containing `*:*`.
- fq:** A text box with red minus and green plus icons on the right.
- sort:** A text box.
- start, rows:** Two text boxes containing `0` and `10`.
- fl:** A text box.
- df:** A text box.
- Raw Query Parameters:** A text box containing `key1=val1&key2=val2`.
- wt:** A dropdown menu with `-----` selected. This dropdown is highlighted with a red rectangle.
- indent off:** A checkbox.

Response format (json, php, csv, and others).

There are more than 16 different formats

Searching for documents



A query can be also submitted using cURL or pointing the web page:

```
$ curl http://localhost:8983/solr/films_small/select?q=*:*
```

Facets

Facets



If enabled, a Solr query can return the number of documents having a fixed value in a field.

This is similar to **count** clause and **group by** clause in SQL.

For example, suppose we want to have the number of books published by each authors

Facets

showing 1 - 10 of 5000 sort by relevance

refine this search by...

- main ingredient ›
- meal/course ›
- cuisine ›
- dietary consideration**
- type
- holiday/celebration
- preparation method

part of menu (173)

Sponsored Recipe

 Chicken & Broccoli Alfredo
Campbell's Kitchen™ / December, 2013

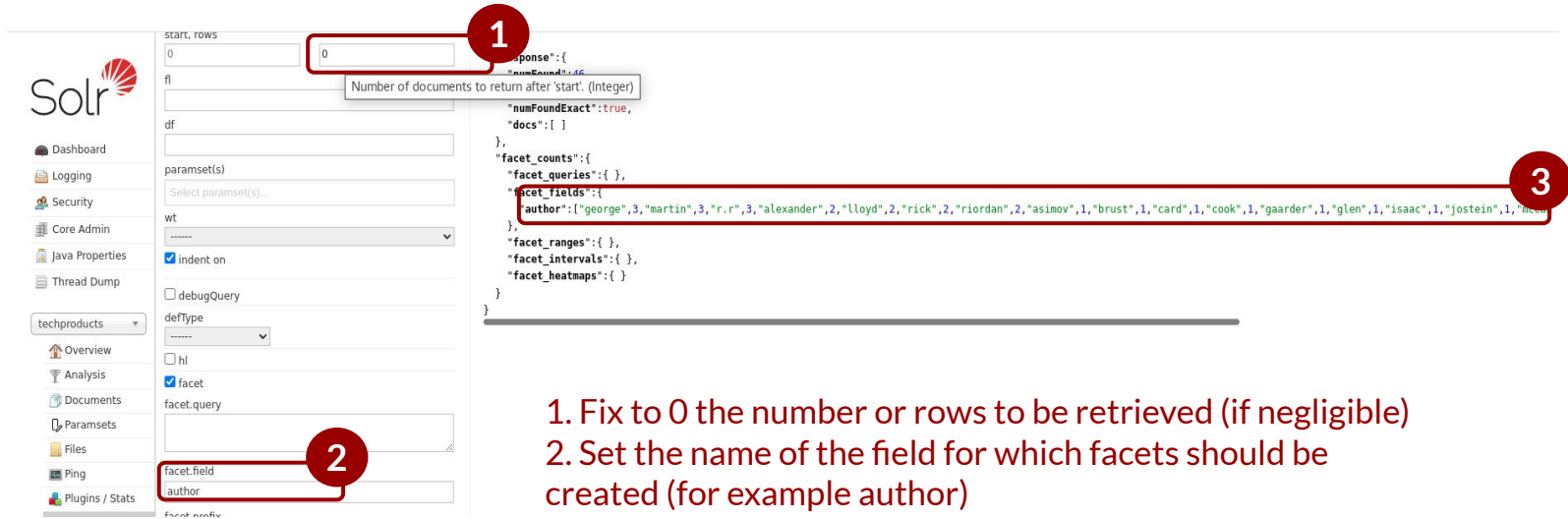


Facet

select from the following:

Vegetarian (1317)	Low Fat (249)	Low Sodium (74)
Healthy (941)	Low/No Sugar (132)	Kosher (53)
High Fiber (413)	Low Carb (124)	Raw (33)
Low Cal (396)	Low Cholesterol (109)	Kosher for Passover (13)
Vegan (395)	Wheat/Gluten-Free (103)	

Facets



The screenshot displays the Solr Admin interface for configuring facets. On the left, the 'techproducts' collection is selected. The 'facet.field' is set to 'author'. The 'start, rows' field is set to 0. The 'response' JSON shows the resulting facets for the 'author' field.

1. Fix to 0 the number or rows to be retrieved (if negligible)

2. Set the name of the field for which facets should be created (for example author)

3. Facets are shown in the result

Facets

Many other parameters are available:

- **facet limit:** number of facets to be retrieved
- **facet sort:** how to sort facets (by index or by count)
- **facet mincount:** the minimum value to make the facet visible

☒ facet

facet.query

facet.field

facet.prefix

facet.contains

☐ facet.contains.ignoreCase

facet.limit

facet.matches

facet.sort

----- ▾

facet.mincount

☐ facet.missing

Facets: partitioning

Facets can also work with numbers and dates.

For example we can create facets as partitions of the field price;

☒ facet

start, rows

Raw Query Parameters



rows = 0: no data, just facets

facet.range=price: name of the field to work on

facet.range.start=0: the lower bound of the ranges

facet.range.end=100: the upper bound of the ranges

facet.range.gap=10: The span of each range expressed as a value to be added to the lower bound

Facets: partitioning

Another example: If we want to partition films by year starting with 25 years ago (our earliest release date is in 2000) and ending today

☒ facet

start, rows

Raw Query Parameters



rows = 0: no data, just facets

facet.range=initial_release_date: name of the field to work on

facet.range.start=NOW/YEAR-25YEAR: the lower bound of the ranges

facet.range.end=NOW: the upper bound of the ranges

facet.range.gap=+1YEAR: The span of each range expressed as a value to be added to the lower bound

Facets: pivoting

Another faceting type is pivot facets, also known as "**decision trees**", allowing two or more fields to be nested for all the various possible combinations.

This can be used, for example, when we can to see how many of the films in the "Drama" category (the genre field) are directed by a director

☒ facet

start, rows

Raw Query Parameters



facet.pivot=genre,directed_by_str

Schema

Schema



The structure of a core/collection is defined by a schema

A schema is an xml file that tells Solr how to ingest documents into the core, process them and spit out an index.

The schema can be managed by hand or in automatic way

We use the term “schemaless” to talk about the automatic way

Schemaless configuration



Schemaless configuration is used just for speed and not accuracy.

Using the schemaless configuration, the structure of a core is managed by Solr.

When managed by the system (Schemaless), Field Types are recognized automatically during ingestion.

As we are going to see later, sometimes schemaless configuration does not make the right choice, and something can go wrong

The default configuration creates a schemaless core.

Field Guessing



In Schemaless mode, during document ingestion, Solr attempts to guess what type of data is in a field while it's indexing it

It also automatically creates new fields in the schema for new fields that appear in incoming documents.

The behaviour of managing data in schemaless mode is defined in the configuration file (that we copied from the `_default` configset)

Let's browse the schema



```
curl http://localhost:8983/solr/films_small/schema/fields
```

or using a browser, open the page

```
http://localhost:8983/solr/films_small/schema/fields
```


Let's browse the schema

- fields
 - directed_by
 - genre
 - id
 - initial_release_date
 - name

Field names are the same as the name written inside the posted file

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 0,
    "fields": [
      {
        "name": "_nest_path_",
        "type": "_nest_path_"
      },
      {
        "name": "_root_",
        "type": "string",
        "docValues": false,
        "indexed": true,
        "stored": false
      },
      {
        "name": "_text_",
        "type": "text_general",
        "multiValued": true,
        "indexed": true,
        "stored": false
      },
      {
        "name": "_version_",
        "type": "plong",
        "indexed": false,
        "stored": false
      },
      {
        "name": "directed_by",
        "type": "text_general"
      },
      {
        "name": "genre",
        "type": "text_general"
      },
      {
        "name": "id",
        "type": "string",
        "multiValued": false,
        "indexed": true,
        "required": true,
        "stored": true
      },
      {
        "name": "initial_release_date",
        "type": "pdates"
      },
      {
        "name": "name",
        "type": "text_general"
      }
    ]
  }
}
```

Let's browse the schema

There are some others that are created automatically during ingestion

- fields
 - `_root_`
 - `_text_`
 - `_version_`

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 0,
    "fields": [
      {
        "name": "_nest_path_",
        "type": "_nest_path_",
        {
          "name": "_root_",
          "type": "string",
          "docValues": false,
          "indexed": true,
          "stored": false,
        }
      },
      {
        "name": "_text_",
        "type": "text_general",
        "multiValued": true,
        "indexed": true,
        "stored": false,
      },
      {
        "name": "_version_",
        "type": "plong",
        "indexed": false,
        "stored": false,
      },
      {
        "name": "directed_by",
        "type": "text_general",
      },
      {
        "name": "genre",
        "type": "text_general",
      },
      {
        "name": "id",
        "type": "string",
        "multiValued": false,
        "indexed": true,
        "required": true,
        "stored": true,
      },
      {
        "name": "initial_release_date",
        "type": "pdates",
      },
      {
        "name": "name",
        "type": "text_general"
      }
    ]
  }
}
```

Let's browse the schema

- field types
 - string
 - text_general
 - plong
 - pdates

```
{
  "responseHeader":{
    "status":0,
    "QTime":0,
    "fields":{
      {
        "name": "_nest_path_",
        "type": "_nest_path_",
        {
          "name": "_root_",
          "type": "string",
          "docValues": false,
          "indexed": true,
          "stored": false,
        }
        {
          "name": "text_",
          "type": "text_general",
          "multiValued": true,
          "indexed": true,
          "stored": false,
        }
        {
          "name": "version_",
          "type": "plong",
          "indexed": false,
          "stored": false,
        }
        {
          "name": "directed_by",
          "type": "text_general",
        }
        {
          "name": "genre",
          "type": "text_general",
        }
        {
          "name": "id",
          "type": "string",
          "multiValued": false,
          "indexed": true,
          "required": true,
          "stored": true,
        }
        {
          "name": "initial_release_date",
          "type": "pdates",
        }
        {
          "name": "name",
          "type": "text_general"
        }
      }
    }
  }
}
```

Types and Field Types

Field Types

```
<fieldType name="pdate" class="solr.DatePointField" docValues="true"/>
<fieldType name="pdates" class="solr.DatePointField" docValues="true" multiValued="true"/>
<fieldType name="pdouble" class="solr.DoublePointField" docValues="true"/>
<fieldType name="pdoubles" class="solr.DoublePointField" docValues="true" multiValued="true"/>
<fieldType name="pfloat" class="solr.FloatPointField" docValues="true"/>
<fieldType name="pfloats" class="solr.FloatPointField" docValues="true" multiValued="true"/>
<fieldType name="phonetic_en" class="solr.TextField" indexed="true" stored="false">
```

Fields

```
<field name="_nest_path_" type="_nest_path_" />
<field name="_root_" type="string" docValues="false" indexed="true" stored="false"/>
<field name="_text_" type="text_general" multiValued="true" indexed="true" stored="false"/>
<field name="_version_" type="plong" indexed="false" stored="false"/>
<field name="directed_by" type="text_general" uninvertible="false" indexed="false" stored="true"/>
<field name="genre" type="text_general"/>
<field name="id" type="string" multiValued="false" indexed="true" required="true" stored="true"/>
<field name="initial_release_date" type="pdates"/>
<field name="name" type="text_general" uninvertible="true" indexed="true" stored="true"/>
```

21 field types



fieldType	Description
<code>BinaryField</code>	For binary data.
<code>BoolField</code>	For true or false data.
<code>CollationField</code>	For multiple language processing with Unicode symbols.
<code>CurrencyFieldType</code>	For currencies and exchange rate data.
<code>DateRangeField</code>	For date range calculations.
<code>DatePointField</code>	For date Fields at a point in time.
<code>DoublePointField</code>	For 64-bit floating point data.
<code>ExternalFileField</code>	For data located in an external file.
<code>EnumFieldType</code>	For ordering data using a scheme other than alphabetic or numeric sorting.
<code>FloatPointField</code>	For 32-bit floating point data.
<code>ICUCollationField</code>	For Unicode Field specifications.
<code>IntPointField</code>	For 32-bit signed integers.
<code>LatLonPointSpatialField</code>	For geospatial data, this holds latitudes and longitudes.
<code>LongPointField</code>	For 64-bit signed integers.
<code>PointType</code>	For sorting geospatial data.
<code>PreAnalyzedField</code>	For passing tokens that have already been analyzed.
<code>RandomSortField</code>	For returning search results in random order.
<code>RPT</code>	For geospatial data coordinates. The long form is SpatialRecursivePrefixTreeFieldType .
<code>StrField</code>	For short strings encoded with UTF-8 Unicode and are not analyzed or tokenized.
<code>TextField</code>	For text that includes multiple words to be broken into tokens.
<code>UUIDField</code>	For a Universally Unique Identifier (UUID) Field.

Some field types



- StrField: used for small text that should not be analyzed or tokenized (for example an email address or an url)
- TextField: used for long text that should be split into several terms to index
- BinaryField: for binary data
- BoolField: for true or false data

Let's browse the schema

Who defined the schema?

As said before, each collection is managed in schemaless mode, this means that we let Solr to manage data, so it will modify the schema on its own while posting a new document

```
{
  "responseHeader":{
    "status":0,
    "QTime":0,
    "fields":[{
      "name":"_nest_path_",
      "type":"_nest_path_",
      {
        "name":"_root_",
        "type":"string",
        "docValues":false,
        "indexed":true,
        "stored":false},
      {
        "name":"_text_",
        "type":"text_general",
        "multiValued":true,
        "indexed":true,
        "stored":false},
      {
        "name":"_version_",
        "type":"plong",
        "indexed":false,
        "stored":false},
      {
        "name":"directed_by",
        "type":"text_general"},
      {
        "name":"genre",
        "type":"text_general"},
      {
        "name":"id",
        "type":"string",
        "multiValued":false,
        "indexed":true,
        "required":true,
        "stored":true},
      {
        "name":"initial_release_date",
        "type":"pdates"},
      {
        "name":"name",
        "type":"text_general"}}}]}
```

Let's browse the schema

Solr

Dashboard
Logging
Security
Core Admin
Java Properties
Thread Dump

techproducts

Overview
Analysis
Documents
Paramsets
Files
Ping
Plugins / Stats
Query
Replication
Schema
Segments info

Field: features

Field-Type: org.apache.solr.schema.TextField
P...Gap: 100
Docs: 20

Flags: Indexed Tokenized Stored UnInvertible Multivalued

Properties
Schema
Index

Index Analyzer: org.apache.solr.analysis.TokenizerChain
Query Analyzer: org.apache.solr.analysis.TokenizerChain

Load Term Info

1. Select the collection
2. Select the Schema item
3. Select the field from the schema
4. Get information about the field
5. Steps Solr follows for building index and during query

Some file type properties



Each field is also assigned to many properties.

For example:

- indexed: the field is indexed or not, it will be used during search operations
- stored: the field is stored or not
- default: the default value if no other value is set at index time
- multivalued : is similar to a one-to-many relationship in a database
- required: mandatory fields, it is commonly used for ID fields and structured data (If "true" and the field during indexing is not found, then Solr will not index the document)

The problem using schemaless configuration



Using schemaless configuration, Solr assigns each field to a field type, depending on the corresponding value in the first posted document. This means that not always the selected field type is right.

To understand this, let's start back again, but this time, instead of posting a small file, let's post the whole version.

The problem using schemaless configuration



Let's delete the films collection

```
bin/solr delete -c films
```

and then let's create again the same collection

```
bin/solr create -c films
```

then posting the whole file

```
bin/post -c films example/films/films.csv
```

The problem using schemaless configuration

The ingestion process fails

Let's try to understand why.

Go browsing data

```
[root@vmcms solr-8.6.1]# bin/post -c films example/films/films.csv
java -classpath /usr/local/solr-8.6.1/dist/solr-core-8.6.1.jar -Dauto=
yes -Dc=films -Ddata=files org.apache.solr.util.SimplePostTool example
/films/films.csv
SimplePostTool version 5.0.0
Posting files to [base] url http://localhost:8983/solr/films/update...
Entering auto mode. File endings considered are xml,json,jsonl,csv,pdf
,doc,docx,ppt,pptx,xls,xlsx,odt,odp,ods,ott,otp,ots,rtf,htm,html,txt,l
og
Posting file films.csv (text/csv) to [base]
SimplePostTool: WARNING: Solr returned an error #400 (Bad Request) for
url: http://localhost:8983/solr/films/update
SimplePostTool: WARNING: Response: {
  "responseHeader":{
    "status":400,
    "QTime":16},
  "error":{
    "metadata":[
      "error-class","org.apache.solr.common.SolrException",
      "root-error-class","java.lang.NumberFormatException"],
    "msg":"ERROR: [doc=/en/quien_es_el_senor_lopez] Error adding field
'name='¿Quién es el señor López?' msg=For input string: \"¿Quién es
el señor López?\",
    "code":400}}
SimplePostTool: WARNING: IOException while reading response: java.io.I
OException: Server returned HTTP response code: 400 for URL: http://lo
calhost:8983/solr/films/update
1 files indexed.
COMMITting Solr index changes to http://localhost:8983/solr/films/upda
te...
Time spent: 0:00:00.163
[root@vmcms solr-8.6.1]#
```

The problem using schemaless configuration

Request-Handler (qt)

/select

— common —

q

,

fq

sort

start, rows

0 10

fl

df

http://vmcms.unipv.it:8983/solr/films/select?q=*&fq=1601641339891

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 11,
    "params": {
      "q": "*,*",
      "fq": "1601641339891"
    }
  },
  "response": { "numFound": 5, "start": 0, "numFoundExact": true, "docs": [
    {
      "name": [0.45],
      "directed_by": ["Gary Lennon"],
      "genre": ["Black comedy|Thriller|Psychological thriller|Indie film|Action Film|"],
      "id": "/en/45_2006",
      "initial_release_date": ["2006-11-30T00:00:00Z"],
      "_version_": 1679444924214280192
    }
  ]
}
```

ONLY 5 documents are indexed. Why?

The problem using schemaless configuration

The reason is that the attribute name has been assigned to the pdoubles type instead of the text_general as before.

But why this happened? Let's have a look to the difference between the first (small) file and the second (whole) file

```
"stored":false},
{
  "name": "_text_",
  "type": "text_general",
  "multiValued": true,
  "indexed": true,
  "stored": false},
{
  "name": "_version_",
  "type": "plong",
  "indexed": false,
  "stored": false},
{
  "name": "directed_by",
  "type": "text_general"},
{
  "name": "genre",
  "type": "text_general"},
{
  "name": "id",
  "type": "string",
  "multiValued": false,
  "indexed": true,
  "required": true,
  "stored": true},
{
  "name": "initial_release_date",
  "type": "pdates"},
{
  "name": "name",
  "type": "pdoubles"}}
```

The problem using schemaless configuration

```
[root@vmcms films]# head -2 films_small.csv
name,directed_by,genre,type,id,initial_release_date
Happy Ero Christmas, Lee Geon-dong, Romance Film|Comedy|East Asian cinema|World cinema,
,/en/happy_ero_christmas,2003-12-17
[root@vmcms films]#
```

```
[root@vmcms films]# head -2 films.csv
name,directed_by,genre,type,id,initial_release_date
.45,Gary Lennon,Black comedy|Thriller|Psychological thriller|Indie film|Action Film|C
rime Thriller|Crime Fiction|Drama,,/en/45_2006,2006-11-30
[root@vmcms films]#
```

The name of the first film in the whole file is “.45”, so Solr assigns pdoubles type to the field, but when it tries to import the 6th document (having a string name) the exceptions is risen.

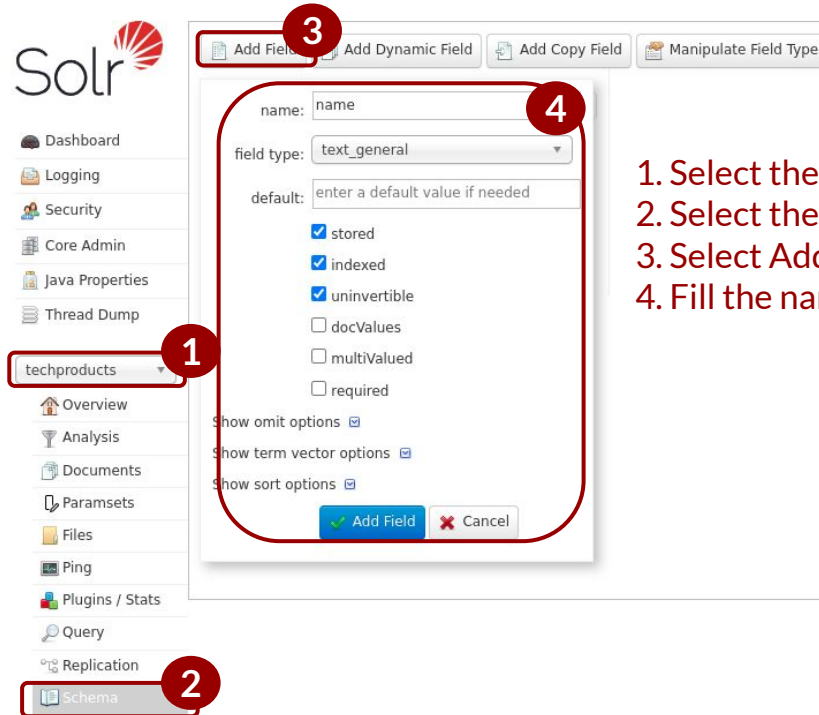
The problem using schemaless configuration



How to solve the problem?

Solution: dropping the field and creating a new one having the right type, by using web console.

Adding new fields using GUI



1. Select the collection
2. Select the Schema item
3. Select Add Field button
4. Fill the name (name) and the type (text_general)

Adding new fields using GUI



Then Posting back again the whole file:

```
bin/post -c films example/films/films.csv
```

Now 1100 documents are ingested.

Adding new fields using CURL



Graphical User Interface offers less control over the properties of your field.

For a fine-grained control, CURL must be used:

```
curl -X POST -H 'Content-type:application/json'
--data-binary '{"add-field": {"name":"name",
"type":"text_general", "multiValued":false,
"stored":true}}' http://localhost:8983/solr/films/schema
```

Customized configuration



Three main configuration files:

1. `server/solr/solr.xml`: server instance configurations
2. `server/solr/corename/conf/solrconfig.xml`: collection configuration
3. `server/solr/corename/conf/managed-schema`: schema configuration (it defines how documents should be processed during ingestion)

These last two files are copied from `server/solr/configsets/_default/conf/`

Disabling automatic field creation



1. Open `server/solr/corename/conf/solrconfig.xml`:
2. Turn to false the option `autoCreateFields`
3. Restart Solr

You can also use API:

```
curl http://localhost:8983/solr/config -d
'{"set-user-property": {"update.autoCreateFields":
"false"}}'
```

Indexed fields



Some attributes are indexed, some others are not.

```
{
  "name": "_text_",
  "type": "text_general",
  "multiValued": true,
  "indexed": true,
  "stored": false},
{
  "name": "_version_",
  "type": "plong",
  "indexed": false,
  "stored": false},
{
  "name": "directed_by",
  "type": "text_general"},
{
  "name": "genre",
  "type": "text_general"},
}
```

Indexed fields



Indexed fields are important because they can be used for searching operation.

Let's have a try, using the UI:

- select the `directed_by` attribute and delete the “copied to” attribute
- drop the `directed_by` attribute using GUI
- create the `directed_by` attribute (`text_general`) using GUI with **`indexed=false` and `univertible=false`**
- reload the films file to reindex data
- try to look for those films which have been directed by Spike Lee: nothing is found

Indexed fields

Request-Handler (qt)

/select

— common —

q
directed_by:*

fq

sort

start, rows
0 10

fl

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 0,
    "params": {
      "q": "directed_by:*",
      "_": "1601975272860"
    }
  },
  "response": { "numFound": 0, "start": 0, "numFoundExact": true, "docs": [] }
}
```


Copy Fields



Problem: how we can run a query for looking for a value in several attributes (let's say in genre and in name)?

Solution: using copy fields

A copy field is a special field used to duplicate data entered into a second field (which can be more than one)

Copy Fields

Let's see how to create a copy field:

- Using GUI, select Schema
- Select Add Copy Field
- Source = * (all attributes)
- Destination = _text_
- Reindex data
- Search data



 Dashboard

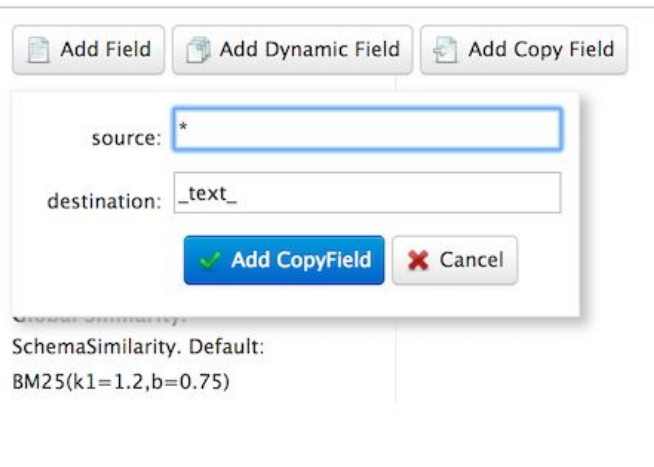
 Logging

 Cloud

 Collections

 Java Properties

 Thread Dump



The image shows a screenshot of the Solr GUI. On the left is a sidebar with navigation links: Dashboard, Logging, Cloud, Collections, Java Properties, and Thread Dump. The main area displays three buttons at the top: "Add Field", "Add Dynamic Field", and "Add Copy Field". The "Add Copy Field" button is selected, opening a modal dialog. Inside the dialog, the "source:" field contains an asterisk (*) and the "destination:" field contains "_text_". At the bottom of the dialog are two buttons: "Add CopyField" (with a green checkmark icon) and "Cancel" (with a red X icon). Below the dialog, the text "SchemaSimilarity. Default: BM25(k1=1.2,b=0.75)" is visible.

Copy Fields



What this does is make a copy of all fields and put the data into the "_text_" field.

It can be very expensive to do this with your production data because it tells Solr to effectively index everything twice. It will make indexing slower, and make your index larger. With your production data, you will want to be sure that only copy fields that really warrant it for your application are indexed.

Copy Fields

Request-Handler (qt)

/select

— common —

q

Spike Lee

fq

sort

start, rows

0 10

fl

df

We don't define any field, because by the default configuration, Solr uses `_text_` attribute. If we want to use a different attribute, then select the `df` (default search field) parameter

```
"_": "1601975272860"}},  
"response": {"numFound": 11, "start": 0, "numFoundExact": true, "docs": [  
  {  
    "name": ["25th Hour"],  
    "directed_by": ["Spike Lee"],  
    "genre": ["Crime Fiction|Drama"],  
    "id": "/en/25th_hour",  
    "initial_release_date": ["2002-12-16T00:00:00Z"],  
    "_version_": 1679812935438827522},  
  {  
    "name": ["Bamboozled"],  
    "directed_by": ["Spike Lee"],  
    "genre": ["Satire|Indie film|Music|Black comedy|Comedy-drama|Med  
    "id": "/en/bamboozled",  
    "initial_release_date": ["2000-10-06T00:00:00Z"]
```

Copy Fields



Let's use

```
curl http://localhost:8983/solr/films/schema/copyfields
```

Vectors

Using Vectors



Now, let's suppose we want to assign each indexed document to a vector, representing the document, and we want to use it to compute similarities.

Step 1: create a new core named films2

```
./bin/solr create -c films2
```

Using Vectors

Step 2: create a new field type

1. Select Schema
2. Select Manipulate Field Type
3. Fill the field with the new field type named `knn_vector_10`
4. Add Field Type

The screenshot shows the Solr Admin interface. On the left is a sidebar menu with the following items: Dashboard, Logging, Security, Core Admin, Java Properties, Thread Dump, films (selected), Overview, Analysis, Documents, Paramsets, Files, Ping, Plugins / Stats, Query, and Replication. The 'Schema' item at the bottom of the sidebar is highlighted with a red box and a red circle containing the number 1. The main content area shows a 'Manipulate Field Type' button at the top right, highlighted with a red box and a red circle containing the number 2. Below this is a 'Add FieldType Template' dialog box. The dialog has a text area containing the following JSON configuration:

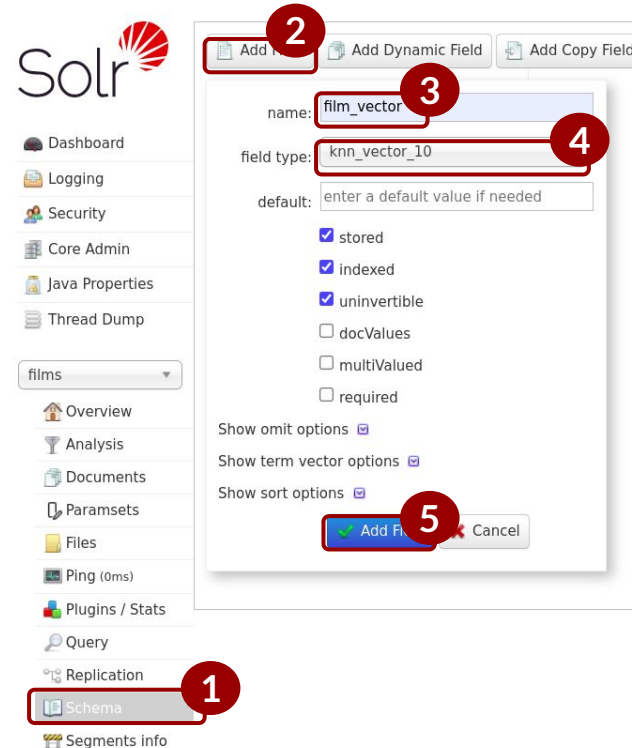
```
{  "add-field-type" : {    "name": "knn_vector_10",    "class": "solr.DenseVectorField",    "vectorDimension": 10,    "similarityFunction": "cosine",    "knnAlgorithm": "hnsf"  }}
```

 The text area is highlighted with a red box and a red circle containing the number 3. At the bottom of the dialog, there is an 'Add FieldType' button (highlighted with a red box and a red circle containing the number 3) and a 'Cancel' button.

Using Vectors

Step 3: create a new field having type
knn_vector_10

1. Select Schema
2. Add Field
3. Set the field name
4. Set the field type
5. Add the new field



Using Vectors

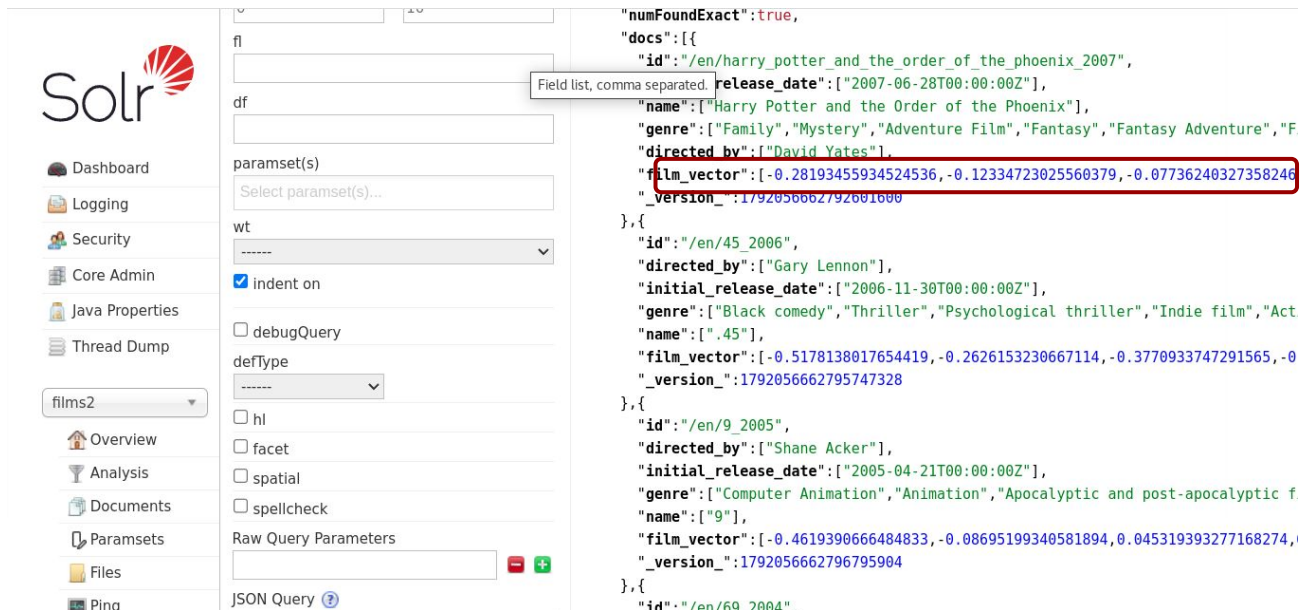


Step 4: index all documents

```
./bin/post -c films2 example/films/films2.json
```

Using Vectors

Step 5: query for all films



Solr

Dashboard
Logging
Security
Core Admin
Java Properties
Thread Dump

films2

Overview
Analysis
Documents
Paramsets
Files
Ping

fl
df
paramset(s)
wt
indent on
debugQuery
defType
hl
facet
spatial
spellcheck
Raw Query Parameters
JSON Query

```
{
  "numFoundExact": true,
  "docs": [
    {
      "id": "/en/harry_potter_and_the_order_of_the_phoenix_2007",
      "release_date": ["2007-06-28T00:00:00Z"],
      "name": ["Harry Potter and the Order of the Phoenix"],
      "genre": ["Family", "Mystery", "Adventure Film", "Fantasy", "Fantasy Adventure", "F"],
      "directed_by": ["David Yates"],
      "film_vector": [-0.28193455934524536, -0.12334723025560379, -0.07736240327358246, ...],
      "_version_": 1792056662792601600
    },
    {
      "id": "/en/45_2006",
      "directed_by": ["Gary Lennon"],
      "initial_release_date": ["2006-11-30T00:00:00Z"],
      "genre": ["Black comedy", "Thriller", "Psychological thriller", "Indie film", "Act"],
      "name": [".45"],
      "film_vector": [-0.5178138017654419, -0.2626153230667114, -0.3770933747291565, -0.1792056662795747328],
      "_version_": 1792056662795747328
    },
    {
      "id": "/en/9_2005",
      "directed_by": ["Shane Acker"],
      "initial_release_date": ["2005-04-21T00:00:00Z"],
      "genre": ["Computer Animation", "Animation", "Apocalyptic and post-apocalyptic f"],
      "name": ["9"],
      "film_vector": [-0.4619390666484833, -0.08695199340581894, 0.045319393277168274, ...],
      "_version_": 1792056662796795904
    },
    {
      "id": "/en/69_2004"
    }
  ]
}
```

Using Vectors



Now, suppose we want to search for all those films that are similar to these three:

- Finding Nemo
- Bee Movie
- Harry Potter and the Chamber of Secrets

We start building the vector as the resulting average vector computed from the vectors representing all three films:

`[-0.1784, 0.0096, -0.1455, 0.4167, -0.1148, -0.0053, -0.0651, -0.0415, 0.0859, -0.1789]`

Using Vectors



Then we search for 10 documents having a vector as closer as possible to the resulting average vector. We use JSON Query:

```
{
  "query" : "{!knn f=film_vector topK=10}[-0.1784, 0.0096,
-0.1455, 0.4167, -0.1148, -0.0053, -0.0651, -0.0415,
0.0859, -0.1789]",
  "fields" : "name"
}
```

Using Vectors

Solr

Dashboard

Logging

Security

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Documents

Paramssets

Files

Ping

Plugins / Stats

Query

Replication

Schema

df

paramset(s)

Select paramset(s)...

wt

☒ indent on

☐ debugQuery

deftype

☐ hl

☐ facet

☐ spatial

☐ spellcheck

Raw Query Parameters

JSON Query ?

topK=10}{-0.1784, 0.0096,
-0.1455, 0.4167, -0.1148, -0.0053,

Execute Query

```
"docs":[{"name":["Finding Nemo"]}, {"name":["Bambi II"]}, {"name":["Cats & Dogs"]}, {"name":["Bee Movie"]}, {"name":["Fantastic Mr. Fox"]}, {"name":["Curious George"]}, {"name":["Eight Below"]}, {"name":["Chicken Little"]}, {"name":["Bolt"]}, {"name":["Batman: Mystery of the Batwoman"]}]}
```

Analizers

Analyzers



Now we want to understand how Solr can index and look for data

During ingestion, Solr indexes data

During searching, Solr retrieves data

On both times, Solr makes an analysis on the data

We can define the analysis Solr should do during indexing and during inquiring

Analyzers



The analysis process takes two phases:

1. Tokenization: the data is split into terms
2. Filtering: all terms are filtered and transformed (when needed)

The tokenization and filtering processes are guided by rules and depend on the field type.

So we define a set of rules for each field type that Solr follows during analysis process

Analyzers



Then Solr defines two Analyzers:

- one for indexing
- one for queries

Each Analyzer defines;

- One Tokenizer
- One or More Filters

Example of Analyzer



```
<fieldType name="text_general" class="solr.TextField" positionIncrementGap="100" multiValued="true">
  <analyzer type="index">
    <tokenizer name="standard"/>
    <filter name="stop" ignoreCase="true" words="stopwords.txt"/>
    <filter name="lowercase"/>
  </analyzer>
  <analyzer type="query">
    <tokenizer name="standard"/>
    <filter name="stop" ignoreCase="true" words="stopwords.txt"/>
    <filter expand="true" ignoreCase="true" synonyms="synonyms.txt" name="synonymGraph"/>
    <filter name="lowercase"/>
  </analyzer>
</fieldType>
```

Some Tokenizers

Name and Class	Tokenizer Actions
Classic Tokenizer <code>ClassicTokenizerFactory</code>	• Keeps domain names and email addresses as a token • Words with hyphens and only letters are split into two tokens • Words with hyphens and numbers are retained as one token • Periods that are not followed by whitespace are kept
Keyword Tokenizer <code>KeywordTokenizerFactory</code>	• Keeps the entire text field as one token • Well suited for id fields and structured data
Letter Tokenizer <code>LetterTokenizerFactory</code>	• Retains continuous letters only • All other characters are removed
Lowercase Tokenizer <code>LowerCaseTokenizerFactory</code>	• Retains continuous letters only • All other characters removed • Makes all uppercase characters to lowercase

Some Tokenizers

Name and Class	Tokenizer Actions
Standard Tokenizer <code>StandardTokenizerFactory</code>	• Treats whitespace and punctuation (not numbers) as delimiters • Periods that are not followed by whitespace are retained • Domain names are preserved but email addresses are not • Words with hyphens are split into two tokens
UAX29 URL Email Tokenizer <code>UAX29URLEmailTokenizerFactory</code>	• Treats whitespace and punctuation (not numbers) as delimiters • Periods that are not followed by whitespace are retained • Domain names are preserved but email addresses are not • For words with letters only, hyphens are split into two tokens • For words with letters and numbers, hyphens are split into two tokens • Preserves internet domain names, email addresses, full paths to file://, http(s)://, ftp:// addresses, and IPv4 and IPv6 addresses
White Space Tokenizer <code>WhiteSpaceTokenizerFactory</code>	• Uses whitespace only as delimiters • Returns everything else as-is, including punctuation

Some Tokenizers



Each Tokenizer is defined by a class. For example the Standard Tokenizer is defined by the class `solr.StandardTokenizerFactory`.

By using this class, a token is built splitting the text considering the white space and punctuation

For a full list of Tokenizers see

<https://solr.apache.org/guide/solr/latest/indexing-guide/tokenizers.html>

Some Filters

Name and Class	Filtering Actions
Classic Filter <code>ClassicFilterFactory</code>	• Removes possessive "'s" • Removes periods in acronyms • Removes periods at the end of a sentence
English Minimal Stem Filter <code>EnglishMinimalStemFilterFactory</code>	• Removes plural "s" to create singular form
English Possessive Filter <code>EnglishPossessiveFilterFactory</code>	• Removes possessive "'s"
Hyphenated Words Filter <code>HyphenatedWordsFilterFactory</code>	• Combines words that are hyphenated
Keep Word Filter <code>KeepWordFilterFactory</code>	• Retains only the tokens that are listed in a file identified with the argument words="keepwords.txt" • Using the argument ignoreCase="true/false" with the default "false" assumes cases matter, and "true" assumes all keywords are lowercased.

Some Filters

Name and Class	Filtering Actions
KStem Filter <code>KStemFilterFactory</code>	• Applies the KStem stemming algorithm for words in English • Less aggressive than the Porter stemmer
Lowercase Filter <code>LowerCaseFilterFactory</code>	• Changes all uppercase letters to lowercase
Porter Stem Filter <code>PorterStemFilterFactory</code>	• Applies the Porter stemming algorithm for words in English • More aggressive than the KStem stemmer
Stop Filter <code>StopFilterFactory</code>	• Removes tokens listed in a file identified with the argument words="stopwords.txt" • Using the argument ignoreCase="true/false" with the default "false" assumes cases matter, and "true" assumes all stopwords are lowercased.

Some Filters



Each Filter is defined by a class. For example the Dropping Stop Words Filter is defined by the class `StopFilterFactory`. This class identifies all those words not to index by file `stopword.txt` (each field can be set to use a different stopwords file, see `conf/lang/stopword_xx.txt`).

Example of Stop Words: "a", "an", "the", "and", "in", "on"

Some Filters



Other interesting Filtering classes are `solr.PorterStemFilterFactory` and `solr.SnowballPorterFilterFactory` (for english) or `solr.ItalianLightStemFilterFactory` (for italian)

Some Filters



For Synonym expansion a specific class is available:

`solr.SynonymGraphFilterFactory`

This class adds synonyms at the same position in the document as the original term using an external synonyms file

For more information about Filters see:

<https://solr.apache.org/guide/solr/latest/indexing-guide/filters.html>

What keep in mind



We can define more than one filters but

1. the order matters
2. you should put the more general filters near the start

Analyzer in action



- each field is assigned to a field type
- each field type is assigned to an analyzer process
- an analyzer defines a sequence of tokenization and filtering steps
- different sequence can yield different results

Analyzer in action

The screenshot displays the Solr Admin interface with the following components and annotations:

- Left Sidebar:** Contains navigation links. The **Schema** link is highlighted with a red box and a red circle containing the number 2.
- Field Selection:** In the 'Field' section, the **films** field is selected, highlighted with a red box and a red circle containing the number 1.
- Field Details:** The details for the **films** field are shown, including its type (**text_en**) and field-type (**org.apache.solr.schema.TextField**). The **Dynamic Field / * _txt_en** section is also visible.
- Analyzer Configuration:** The **Index Analyzer** and **Query Analyzer** are both set to **org.apache.solr.analysis.TokenizerChain**. This configuration is highlighted with a red box and a red circle containing the number 4.
- Buttons:** At the top, there are buttons for **Add Field**, **Add Dynamic Field**, and **Add Copy Field**. Below the field details, there are buttons for **delete field** and **delete dynamic field**.

1. Select the core
2. Select Schema
3. Select the field
4. Analyzers for Indexing and Query

Analyzer in action

Field-Type: org.apache.solr.schema.TextField

Index Analyzer: org.apache.solr.analysis.TokenizerChain

Tokenizer: org.apache.lucene.analysis.standard.StandardTokenizerFactory

class: solr.StandardTokenizerFactory
luceneMatchVersion: 8.6.1

Token Filters: org.apache.lucene.analysis.core.StopFilterFactory

words: lang/stopwords_en.txt
class: solr.StopFilterFactory
ignoreCase
luceneMatchVersion: 8.6.1

org.apache.lucene.analysis.core.LowerCaseFilterFactory

class: solr.LowerCaseFilterFactory
luceneMatchVersion: 8.6.1

org.apache.lucene.analysis.en.EnglishPossessiveFilterFactory

class: solr.EnglishPossessiveFilterFactory
luceneMatchVersion: 8.6.1

org.apache.lucene.analysis.miscellaneous.KeywordMarkerFilterFactory

protected: protwords.txt
class: solr.KeywordMarkerFilterFactory
luceneMatchVersion: 8.6.1

org.apache.lucene.analysis.en.PorterStemFilterFactory

class: solr.PorterStemFilterFactory
luceneMatchVersion: 8.6.1

5

Expand to show all analyzer steps:
1 tokenizer step + 5 filtering steps

ST: split the sentence in tokens

SF: remove unneeded tokens (such as articles)

LCF: lower case all tokens

EPF: remove english possessive adjectives and pronouns

KMF: identify those tokens that should not be stemmed

PSF: identify those tokens that should be stemmed

Analyzer in action

The screenshot shows the Elasticsearch Analyzer web interface. On the left is a sidebar with navigation links: Core Admin, Java Properties, Thread Dump, a dropdown menu showing 'films', Overview, Analysis (highlighted with a red box and a red circle with the number 1), Dataimport, Documents, Files, Ping, Plugins / Stats, Query, Replication, Schema, and Segments info. The main area is divided into two columns. The left column has a 'Field Value (Index)' text box containing the sentence 'the red fox jumped over the lazy brown dog' (highlighted with a red box and a red circle with the number 2). Below this is a dropdown menu for 'Analyse Fieldname / FieldType' with 'text_en' selected (highlighted with a red box and a red circle with the number 3). The right column has a 'Field Value (Query)' text box, a checked 'Verbose Output' checkbox, and a blue 'Analyse Value' button (highlighted with a red box and a red circle with the number 4). Below the interface, a list of four steps is provided: 1. Select Analysis, 2. Write a sentence, 3. Select the field type, and 4. Start Analysis.

1. Select Analysis

2. Write a sentence

3. Select the field type

4. Start Analysis

Analyzer in action

Core Admin

Java Properties

Thread Dump

films

Overview

Analysis

Dataimport

Documents

Files

Ping

Plugins / Stats

Query

Replication

Schema

Segments info

Field Value (Index)

The red fox jumped OVER the lazy brown Dog!

Analyse Fieldname / FieldType: text_en

Schema Browser

ST	The	red	fox	jumped	OVER	the	lazy	brown	Dog
SF		red	fox	jumped	OVER		lazy	brown	Dog
LCF		red	fox	jumped	over		lazy	brown	dog
EPF		red	fox	jumped	over		lazy	brown	dog
SKMF		red	fox	jumped	over		lazy	brown	dog
PSF		red	fox	jump	over		lazi	brown	dog

Field Value (Query)

☐ Verbose Output

Analyse Values

ST: tokenization

SF: articles are removed

LCF: lower case

EPF: remove english possessive adjectives and pronouns

KMF: identify those tokens that should not be stemmed

PSF: identify those tokens that should be stemmed

Last line shows all the tokens that are indexed

Computing Similarities



Solr makes use of different methods for computing similarities documents and query. The two used the most are:

1. ClassicSimilarityFactory: it combines TF-IDF and the cosine similarity
2. BM25Similarity: it is the predefined method for computing similarity based on the Best Match 25 Algorithm

The method for computing similarity is defined into the schema.xml file

The BM25 score function

$$score(q, d) = \sum_{i=1}^{|q|} idf(q_i) \cdot \frac{tf(q_i, d) \cdot (k_1 + 1)}{tf(q_i, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

$$idf(q_i) = \log \frac{N - df(q_i) + 0.5}{df(q_i) + 0.5}$$

q_i = i^{th} term of the query

tf = term frequency (number of times a particular term appears in a document)

|d| = length of the document in words (terms)

avgdl = the average document length over all the documents of the collection

k_1 , **b** = free parameters, usually chosen as $k_1 = 2.0$ and $b = 0.75$

N = total number of documents in the collection

df(qi) = the number of documents containing the query term q_i

Computing Similarities



To be added to managed-schema.xml

```
<similarity class="solr.BM25SimilarityFactory">
  <!-- start with the defaults -->
    <float name="k1">1.2</float>
    <float name="b">0.75</float>
</similarity>
```

Replication and High Availability

Solr: advanced configuration



So far, we have used Solr in standalone mode.

In a common scenario, you might have too much data, too much query to execute, that a single Solr server is unable to handle the entire workload

Solution: scaling up the infrastructure by building a cluster of server Solr

Requests are distributed across several instances

SolrCloud



In order to create a cluster of Solr instances, SolrCloud should be used

SolrCloud is designed to build a highly available, fault tolerant environment

To get this aim, SolrCloud makes use of two different techniques:

- Sharding
- Replication

Sharding



The basic idea is that a collection is split into pieces (also called shards) each of them hosted on multiple machines

So, each shard contains a subset of all documents in the collection

Incoming queries are distributed to every shard in the collection, which respond with merged results.

Replication



Allows you to add servers with additional copies of your collection to handle higher concurrent query load by spreading the requests around to multiple machines.

Sharding and replication are not mutually exclusive, and together make Solr an extremely powerful and scalable platform.

Zookeeper



As an instance needs to read the configuration files and all instances can be distributed even geographically, we need a way to make the configuration available to all instances, even when it changes (for example because a new field is added to the schema)

Zookeeper is tool embedded in SolrCloud which is able to synchronize configuration among different instances

Zookeeper



Of course we need only one Zookeeper instance, regardless the number of instances belonging to the cluster

All instances interact with the same Zookeeper instance.

The scenario



We have two servers, named ServerA and ServerB

We want to run 3 SorlCloud instances on each server (for a total of 6 instances)

The zookeeper instance is executed on the first instance of the ServerA

All other five instances will connect to zookeeper running on ServerA

Building the cluster



On Server A

let's start the first instance

```
./bin/solr start -cloud
```

The command start Solr in Cloud Mode

The default port is 8983

The embedded Zookeeper is started on port 9983

Building the cluster



On Server A

let's start the second instance

```
./bin/solr start -cloud -p 7574 -z server_a:9983
```

The command start a new instance in cloud mode

The instance in run on port 7574

The instance is connected to server A on port 9983

Building the cluster



On Server A

let's start the third instance

```
./bin/solr start -cloud -p 5756 -z server_a:9983
```

The command start a new instance in cloud mode

The instance in run on port 5756

The instance is connected to server A on port 9983

Building the cluster

Let's see what is running on Server A:

```
./bin/solr status
```

```
Solr process 3599 running on port 5756
{
  "solr_home":"/usr/local/solr-8.6.1/server/solr",
  "version":"8.6.1 6ella1c3f0599f1c918bc69c4f51928d23160e99",
  "startTime":"2020-10-08T10:08:01.560Z",
  "uptime":"0 days, 0 hours, 0 minutes, 41 seconds",
  "memory":"51.5 MB (%10.1) of 512 MB",
  "cloud":{
    "ZooKeeper":"localhost:9983",
    "liveNodes":"3",
    "collections":"0"}}}
```

Found 3 Solr nodes:

Solr process 2860 running on port 8983

```
{
  "solr_home":"/usr/local/solr-8.6.1/server/solr",
  "version":"8.6.1 6ella1c3f0599f1c918bc69c4f51928d23160e99",
  "startTime":"2020-10-08T10:01:47.925Z",
  "uptime":"0 days, 0 hours, 6 minutes, 53 seconds",
  "memory":"128.8 MB (%25.1) of 512 MB",
  "cloud":{
    "ZooKeeper":"localhost:9983",
    "liveNodes":"3",
    "collections":"0"}}}
```

Solr process 3331 running on port 7574

```
{
  "solr_home":"/usr/local/solr-8.6.1/server/solr",
  "version":"8.6.1 6ella1c3f0599f1c918bc69c4f51928d23160e99",
  "startTime":"2020-10-08T10:06:41.327Z",
  "uptime":"0 days, 0 hours, 2 minutes, 0 seconds",
  "memory":"59.5 MB (%11.6) of 512 MB",
  "cloud":{
    "ZooKeeper":"localhost:9983",
    "liveNodes":"3",
    "collections":"0"}}}
```


Building the cluster



On Server B

- let's start the all instances
 - `./bin/solr start -cloud -p 8983 -z server_a:9983`
 - `./bin/solr start -cloud -p 7574 -z server_a:9983`
 - `./bin/solr start -cloud -p 5756 -z server_a:9983`

Building the cluster

Please check your logs for more information

Nodes

Free

ZK Status

Graph

Collections

Java Properties

Thread Dump

Suggestions

No collections available
Go and create one

No cores available
Go and create one

Refresh Show all details

Hosts 1 - 2 of 2. Filter by: Host/node name Show 10 hosts per page.

Host	Node	CPU	Heap	Disk usage	Requests	Collections	Replicas
193.204.40.105 <i>Java</i> <i>Load:</i> <i>show details...</i>	5756_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)
	7574_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)
	8983_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)
193.204.40.52 <i>Java</i> <i>Load:</i> <i>show details...</i>	5756_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)
	7574_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)
	8983_solr Uptime: <i>show details...</i>	%	%		RPM: p95: ms	(none)	(none)

By using the Graphical User Interface we can get information about the amount of resources that are used by each instance

Building the cluster



Let's create now a collection, called `shared_films`, with 3 shards and 2 replicas:

```
./bin/solr create -c shared_films -s 3 -rf 2
```

Building the cluster



- **Leader**
- Active
- *Recovering*
- Down
- *Recovery Failed*
- Inactive
- Gone
- **Replica Type:**
 - N(NRT), T(TLOG),
 - P(PULL)

Building the cluster



This means that:

- all documents having a hash in `[00000000 - D554FFFF]` is stored in the shard1
- all documents having a hash in `[D5550000 - 2AA9FFFF]` is stored in the shard2
- all documents having a hash in `[2AAA0000 - 7FFFFFFF]` is stored in the shard3

The health of a collection

```
./bin/solr healthcheck -c shared_films
```

```
[root@vmcms solr-8.6.1]# ./bin/solr healthcheck -c shared_films
{
  "collection": "shared_films",
  "status": "healthy",
  "numDocs": 0,
  "numShards": 2,
  "shards": [
    {
      "shard": "shard1",
      "status": "healthy",
      "replicas": [
        {
          "name": "core_node3",
          "url": "http://193.204.40.52:5756/solr/shared_films_shard1_replica_n3",
          "numDocs": 0,
          "status": "active",
          "uptime": "0 days, 0 hours, 46 minutes, 32 seconds",
          "memory": "212.5 MB (%41.5) of 512 MB",
          "leader": true
        },
        {
          "name": "core_node5",
          "url": "http://193.204.40.52:8983/solr/shared_films_shard1_replica_n5",
          "numDocs": 0,
          "status": "active",
          "uptime": "0 days, 0 hours, 46 minutes, 58 seconds",
          "memory": "97.6 MB (%19.1) of 512 MB",
          "leader": true
        }
      ]
    },
    {
      "shard": "shard2",
      "status": "healthy",
      "replicas": [
        {
          "name": "core_node7",
          "url": "http://193.204.40.52:7574/solr/shared_films_shard2_replica_n7",
          "numDocs": 0,
          "status": "active",
          "uptime": "0 days, 0 hours, 46 minutes, 52 seconds",
          "memory": "241 MB (%47.1) of 512 MB",
          "leader": true
        },
        {
          "name": "core_node8",
          "url": "http://193.204.40.52:5756/solr/shared_films_shard2_replica_n8",
          "numDocs": 0,
          "status": "active",
          "uptime": "0 days, 0 hours, 46 minutes, 32 seconds",
          "memory": "213 MB (%41.6) of 512 MB"
        }
      ]
    }
  ]
}
```